

Guide d'intégration — API Vocale N'ma SIM

Ce guide explique comment intégrer la **compréhension vocale** (reconnaissance des intentions du client en soussou) dans l'application web de la borne.

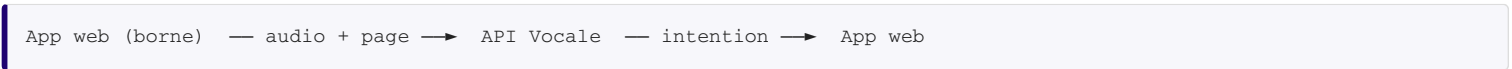
Le fonctionnement est **identique à l'API KYC** : l'app web envoie des données à l'API, l'API renvoie un résultat JSON.

0. Origine technique

Cette API reproduit **exactement** la logique du notebook `OpenWakeWord_3Pages_Soussou.ipynb` : extracteur openWakeWord, classifieur `LogisticRegression` (scikit-learn) par page, détection par fenêtre glissante avec sélection de la fenêtre la plus confiante, règle de priorité et rejet. Les modèles (`.pkl`) ont été entraînés avec ce même notebook, sur les données `donnees_soussou`.

1. Ce que fait l'API

Le client parle à la borne (en soussou). L'app web enregistre l'audio et l'envoie à l'API vocale avec le nom de la **page courante**. L'API renvoie l'**intention** détectée.



Exemple : sur la page « type de pièce », le client dit « passeport » → l'API renvoie `passeport` → l'app web sélectionne le passeport et passe à l'étape suivante.

2. Installation et lancement

Prérequis

- Python 3.10 ou 3.11
- Les fichiers du dossier `nma_vocal_api/` (dont le sous-dossier `modeles/`)

Installation

```
cd nma_vocal_api
pip install -r requirements.txt
```

Lancement

```
uvicorn app:app --host 0.0.0.0 --port 8100
```

L'API démarre sur `http://localhost:8100` . Au démarrage, elle charge l'extracteur openWakeWord et les 3 modèles (un par page). Les modèles sont légers (~280 Ko chacun).

Vérifier que tout est prêt

Ouvrir dans le navigateur : `http://localhost:8100/sante`

```
{
  "statut": "ok",
  "modeles_charges": ["page_choix_du_service", "page_type_de_piece", "page_motif_reactivation"],
  "modeles_manquants": []
}
```

3. L'endpoint principal : `/comprendre`

Méthode : `POST` **URL** : `http://localhost:8100/comprendre` **Format** : `multipart/form-data`

Paramètres à envoyer

Champ	Type	Description
page	texte	La page courante (voir valeurs ci-dessous)
audio	fichier	L'audio enregistré (wav, mp3, webm, ogg... — l'API convertit)

Valeurs possibles pour page

- page_choix_du_service → intentions : nouvelle_sim , reactivation_sim
- page_type_de_piece → intentions : carte_electeur , carte_nationale_identite , passeport
- page_motif_reactivation → intentions : blocage , inactivite , perte

Réponse (JSON)

```
{
  "intention": "passeport",
  "raison": "score_max",
  "scores": {
    "carte_electeur": 0.02,
    "carte_nationale_identite": 0.01,
    "passeport": 0.97
  },
  "niveau_sonore": 0.34,
  "duree_s": 1.6,
  "message": "Intention détectée : passeport"
}
```

Champs de la réponse

Champ	Description
intention	L'intention détectée, ou null si non comprise
raison	score_max , priorite_verbe_action , rejet_score_faible , rejet_ambigu , ou audio_vide
scores	Le score de chaque intention (0 à 1)
niveau_sonore	Niveau max de l'audio (proche de 0 = silencieux)
message	Message lisible pour l'app web

4. Le cas « je n'ai pas compris »

Quand intention vaut null , cela veut dire que l'API n'est pas assez sûre. L'app web doit alors demander au client de répéter, au lieu de deviner.

Trois cas de non-compréhension (champ raison) : - audio_vide : le micro n'a rien capté → « Je n'ai pas bien entendu, répétez » - rejet_score_faible : aucune intention n'est assez claire → « Répétez s'il vous plaît » - rejet_ambigu : deux intentions trop proches → « Répétez s'il vous plaît »

C'est un comportement voulu : mieux vaut redemander que lancer le mauvais service.

5. Comment l'appeler depuis l'app web (JavaScript)

```
async function comprendre(page, blobAudio) {
  const form = new FormData();
  form.append("page", page);
  form.append("audio", blobAudio, "audio.webm");

  const reponse = await fetch("http://localhost:8100/comprendre", {
    method: "POST",
    body: form,
  });
  return await reponse.json();
}

// Exemple d'utilisation après avoir enregistré l'audio du client :
const resultat = await comprendre("page_type_de_piece", monBlobAudio);

if (resultat.intention === null) {
```

```
// Demander au client de répéter (jouer l'audio soussou "répétez")
console.log("Non compris :", resultat.raison);
} else {
  // Déclencher l'action correspondante
  console.log("Le client veut :", resultat.intention);
  // ex: if (resultat.intention === "passeport") { selectionnerPasseport(); }
}
```

Enregistrer l'audio dans le navigateur (rappel)

```
let mediaRecorder, morceaux = [];

async function demarrerEnregistrement() {
  const flux = await navigator.mediaDevices.getUserMedia({ audio: true });
  mediaRecorder = new MediaRecorder(flux);
  morceaux = [];
  mediaRecorder.ondataavailable = e => morceaux.push(e.data);
  mediaRecorder.start();
}

function arreterEnregistrement() {
  return new Promise(resolve => {
    mediaRecorder.onstop = () => resolve(new Blob(morceaux));
    mediaRecorder.stop();
  });
}

// Usage : démarrer quand le client commence à parler, arrêter après ~2-3s,
// puis envoyer le blob à comprendre().
```

6. Brancher intention → action

Pour chaque page, associer chaque intention à l'action de l'app web (ce que faisait le clic du bouton) :

Page	Intention	Action app web
page_choix_du_service	nouvelle_sim	Aller au parcours « nouvelle SIM »
page_choix_du_service	reactivation_sim	Aller au parcours « réactivation »
page_type_de_piece	passeport	Sélectionner passeport
page_type_de_piece	carte_nationale_identite	Sélectionner CNI
page_type_de_piece	carte_electeur	Sélectionner carte d'électeur

En pratique : le vocal **remplace le clic**. Quand l'API renvoie une intention, l'app web déclenche la même chose que le bouton correspondant.

7. Important — comment le client doit parler

Le système reconnaît les **mots-clés** que le client prononce. Pour une reconnaissance fiable, la borne doit **guider** le client à dire le mot-clé, via l'audio soussou déjà intégré. Par exemple :

- Page service : la borne dit « Que voulez-vous faire ? » → le client dit « SIM » / « puce » / « numéro » (nouvelle SIM) ou « réactiver » / « réactivation »
- Page pièce : la borne dit « Quelle pièce avez-vous ? » → le client dit « identité » / « électeur » / « passeport »

Le client dit le **mot** (pas forcément une longue phrase). C'est ce qui rend la reconnaissance fiable.

8. Résumé pour démarrer vite

1. `pip install -r requirements.txt`
2. `uvicorn app:app --host 0.0.0.0 --port 8100`
3. Vérifier `http://localhost:8100/sante`
4. Depuis l'app web : `POST /comprendre` avec `page` + `audio`
5. Lire `intention` dans la réponse → déclencher l'action (ou redemander si `null`)

Pour tester en ligne de commande :

```
python tester_api.py mon_audio.wav page_type_de_piece
```

API cohérente avec l'API KYC (même principe FastAPI). Modèles légers, tournent sur CPU.